CSE 4304-Data Structures Lab. Winter 2021
Date: August 16, 2022
Target Group: All
Topic: Stacks

Instructions:

- Task naming format: fullID_T01L03_2A.c/CPP
- If you find any issues in problem description/test cases, comment in the google classroom.
- If you find any test case that is tricky that I didn't include but others might forget to handle, please comment! I'll be happy to add.
- Use appropriate comments in your code. This will help you to easily recall the solution in the future.
- Obtained marks will vary based on the efficiency of the solution.
- Do not use <bits/stdc++.h> library, add the necessary libraries one after another.
- Do not use any built-in functions or any STL libraries. If the solution is extremely difficult without the usage of such components, discuss it with the instructors.
- Modified sections will be marked with BLUE color.

Group	Tasks
1B	1 3 6 7
2B	1 8 9 10
2A	1 2 4 10
1A	1 2 3 4

Task-1:

Stacks is a linear data structure that follows the Last In First Out (LIFO) principle. The last item to be inserted is the first one to be deleted. For example, you have a stack of trays on a table. The tray at the top of the stack is the first item to be moved if you require a tray from that stack.

The Insertion and Deletion of an element from the stack are a little bit different from the traditional operation. We define the two corresponding operations as Push() and Pop() from the stack.

The first line contains *N* representing the size of the stack. The lines contain the 'function IDs' and the required parameter (if applicable). Function ID 1, 2, 3, 4, 5, and 6 corresponds to push, pop, isEmpty, isFull (assume max size of Stack=5), size, and top. The return type of inEmpty and isFull is Boolean. Stop taking input once given -1.

Input	**Output**
5 3 2 1 10 1 20 5 1 30 6 2 1 40 1 50 4 1 60 4 5 1 60 5 2 6 -1	 True Underflow 10 10 20 2 10 20 30 30 10 20 10 20 40 10 20 40 50 False 10 20 40 50 60 True 5 Overflow 5 10 20 40 50 50

Task 2:

- **HTML** stands for Hyper Text Markup Language · HTML is the standard markup language for creating Web pages · HTML describes the structure of a Web page using different **‘tags’**. Each tag has two portions - ‘opening’ and ‘closing’ having the notations as ‘<tagName>’ and ‘</tagName> respectively.

Given a HTML code, your task is to check that the tags are properly provided or not.
Cases:

- ‘No error’: If all the tags are properly given.
- ‘Error at line ##’: If any error is detected.

Input	**Output**
8 <html> <head> <title> title of webpage </title> </head> <body> This is a paragraph </body> </html>	No error
8 <html> <head> <title> title of webpage </head> </title> <body> This is a paragraph </body> </html> -1	Error at Line 3
9 <html> <head> <title> </title> </head> <body> An unordered HTML list This is a paragraph	Error at line 6

</body> </html> -1	
9 <html> <head> <title> </title> </head> <body> An unordered HTML list This is a paragraph </body> </html> -1	Error at line 7
9 <html> <head> <title> </title> </head> An unordered HTML list This is a paragraph </body> </html> -1	Error at line 7
7 <html> <head> <title> </title> </head> <body> An unordered HTML list This is a paragraph -1	Error at line 7

Note:

- You just have to return the occurrence of the first error detected by your solution.
- For the last test case, there are some opening tags, for which no closing tags were found although the entire code was checked. So we assume that there is an error in the last line.

Task 3:
Suppose, you are given an target-array and an integer n. In each iteration, assume that you have n numbers available in a list (1,2,3,... n).
Your task is to build the target array using the following operations:

- "Push": Reads a new element from the beginning list, and pushes it in the array.
- "Pop": Deletes the last element of the array.
- If the target array is already built, stop reading more elements.

Return a list of the operations needed to build the target-array. If there are multiple valid answers, return any of them.

Input	**Output**	**Explanation**
1 3 3	Push Push Pop Push	Read number 1 and automatically push in the array -> [1] Read number 2 and automatically push in the array then Pop it -> [1] Read number 3 and automatically push in the array -> [1,3]
1 2 3 3	Push Push Push	
1 2 4	Push Push	
2 3 4 4	Push Pop Push Push Push	Push 1, Pop 1, Push 2, Push 3, Push 4
1 4 4	Push Push Push Pop Pop Push **Alternative:** Push Push Pop Push Pop Push	Push 1, Push 2 Push 3, Pop 2, Pop 3, Push 4
1 3 4 6 6	Push Push Pop Push Push Push Pop Push	

Task-4: Next Greater Element

Given a set of numbers, your task is to print the ***Next Greater Element (NGE)*** for every element. The NGE for an element ***x*** is the first greater element on the right side of ***x*** in the array. Elements for which no NGE exists, consider the NGE-value as -1.

Examples:

- For an array, the last element always has NGE as -1.
- For an array sorted in descending order, every element has NGE as -1
- For the input array (4, 5, 2, 25) the NGE for each element is (5, 25, 25, -1).
- For the input array (13, 7, 6, 12) the NGE for each element is (-1, 12, 12, -1}

We can solve this using two simple nested loops! The outer loop picks the elements one by one and the inner loop looks for the first greater element for the element picked by the outer loop. If a greater element is found then that element is printed as the NGE, otherwise, -1 is printed.
But this approach has the worst-case Time complexity is $O(n^2)$. (The worst case occurs when the elements are in descending order. Ill lead us to look for all the elements.)
t wi
We can improve the time complexity to **O(n)** using stacks! Your task is to propose an algorithm to find the NGE of every element using stack in O(n) time.

(P.T.O)

Input:
Each test case can consist of a different number of integers. Every test case will end with a -1 indicating the end of a particular test case (-1 will not be considered as part of the input.)

Output:
Find the NGE of every element using a stack with O(n) worst-case time complexity.

Input	**Output**
4 5 2 25 -1	5 25 25 -1
13 7 6 12 -1	-1 12 12 -1
11 13 21 3 20 -1	13 21 -1 20 -1
12 17 1 5 0 2 2 7 18 25 20 12 5 1 2 -1	17 18 5 7 2 7 7 18 25 -1 -1 -1 -1 2 -1
10 20 30 40 50 -1	20 30 40 50 -1
50 40 30 20 10 -1	-1 -1 -1 -1 -1

Task 06:

Write a program that will take a mathematical expression as input and check whether it is properly parenthesized or not.

The first line of input will take an integer **N** signifying the number of test cases. The next lines will be **N** mathematical expressions. Each input expression may contain any single-digit number (0~9), operators (+ - x /) and any parenthesis ( )/[ ]/{ }.

The output will be Yes/No representing whether it is properly parenthesized or not.

Sample Input	**Sample Output**
8 [5 + (2 x 5) - (7 / 2)] [1 + { 3 x (2 / 3)] } [(1 + 1)] [(1 + 1]) [( )] { } { [( ) ( )] ( ) } ((([5 + (2 x 5) - (7 / 2) 5 + (2 x 5) - (7 / 2)] ( ))) ((( ))	 Yes No Yes No Yes No No No No No

Task 07:

Write a program that will take an expression represented using **postfix notation** as input and evaluate the expression using stack.
Input expression may contain addition (+), subtraction (-), multiplication (*), and division (/) operators and numbers between (0~9).

Sample Input	**Sample Output**
2 345*+6- 225+*1+5-52**	17 100

Task 8: Parentheses checking in a Code

Write a program that will take a block of code consisting of N lines as input and return if there is any error in the code or not. Consider the ()[]{} symbols as valid parenthesis.

There will be two types of messages:

- 'No Error': If the code is compiled successfully.
- 'Error': If the parentheses are in the wrong order or if there is any missing parentheses.

Input	**Output**

5
int main(){
  int x=5, arr[10];
  if(x == 5) return True;
  Else return false;
}
	No Errors.

int x=5, arr[10];
  if(x == 5) return True;
  Else return false;
}5
int main(){{
	Error

5
int main(){
  int x=5, arr[10];
  if(x == 5) return True;
  Else return false;
}}
	Error

5
int main(){
  int x=5, arr[10);
  if(x == 5) return True;
  Else return false;
}
	Error

7
for (i=0; i<10;i++){
  printf("Hello %d", i);
  if(i%2 == 0){
  print("even");
  else{
  print("odd");
	Error

} }	
	Error

Limitation:

We have omitted some cases like,

```
for (i=0; i<10;i++)
  printf(“Hello %d”, i);
  if(i%2 == 0){
    print(“even”);
  else{
    print(“odd”);
  }
}
```

Where the closing curly brace at line-7 is a bit ambiguous. If we follow the trivial procedure, program will think that it is the closing bracket for the if condition and give no error. It is true in a sense. We are limited here with the ability to just check the number of parenthesis and not the correctness of the code. We can’t also check that there i7

```
for (i=0; i<10;i++)
  printf(“Hello %d”, i);
  if(i%2 == 0){
    print(“even”);
  }
  else{
    print(“odd”);
  }
}s no parenthesis provided for the ‘for loop’.
```

Task 9: Remove All Adjacent Duplicates In String

You are given a string *S* consisting of lowercase English letters. A duplicate removal consists of choosing two **adjacent** and **equal** letters and removing them. We repeatedly make duplicate removals on *S* until we no longer can.

Given the string, your task is to return the final string after all such duplicate removals have been made. (It can be proven that the answer is unique).

Input	**Output**	**Explanation**
abbaca	ca	'abbaca' we could remove 'bb' since the letters are adjacent and equal. The result of this move is that the string is 'aaca', of which only 'aa' has to be removed, so the final string is 'ca'.
abbaac	ac	
abbbaaa	aba	
aaabaabaaa	Null	
cxyyyxccc	cxyxc	
geeksfogeegr	gksfor	
datastructure	datastructure	
caaabbbaacdddd	cabc	

Task 10: STPAR - Street Parade

For sure, the love mobiles will roll again on this summer's street parade. Each year, the organizers decide on a fixed order for the decorated trucks. Experience taught them to keep free a side street to be able to bring the trucks into order.
The side street is so narrow that two cars can't pass each other. Thus, the love mobile that enters the side street last must necessarily leave the side street first. Because the trucks and the ravers move up close, a truck cannot drive back and re-enter the side street or the approach street.
You are given the order in which the love mobiles arrive. Write a program that decides if the love mobiles can be brought into the order that the organizers want them to be.

Input
There are several test cases. The first line of each test case contains a single number **n**, the number of love mobiles. The second line contains the numbers ***1 to n*** in an arbitrary order. All the numbers are separated by single spaces. These numbers indicate the order in which the trucks arrive on the approach street. No more than 1000 love mobiles participate in the street parade.

Output
For each test case your program has to output a line containing a single word "yes" if the love mobiles can be re-ordered with the help of the side street, and a single word "no" in the opposite case.

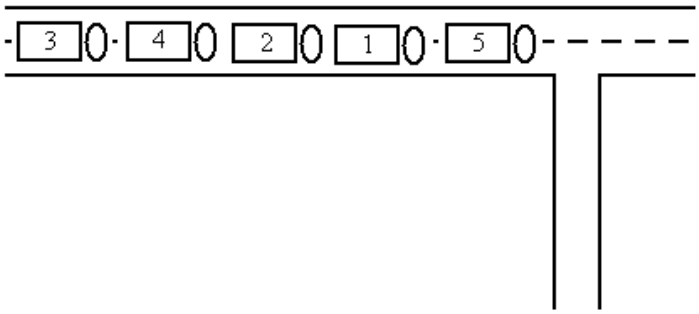
(PTO)

Input	Output
5 5 1 2 4 3	Yes
5 5 3 1 2 4	Yes
5 5 4 3 2 1	Yes
5 1 2 4 3 5	Yes
5 1 2 5 3 4	Yes
5 4 5 1 2 3	No
5 1 4 3 5 2	No
5 4 1 2 3 5	Yes
6 5 2 1 4 3 6	Yes
6 5 2 1 6 4 3	No
5 2 1 5 3 4	Yes
5 1 2 4 5 3	No

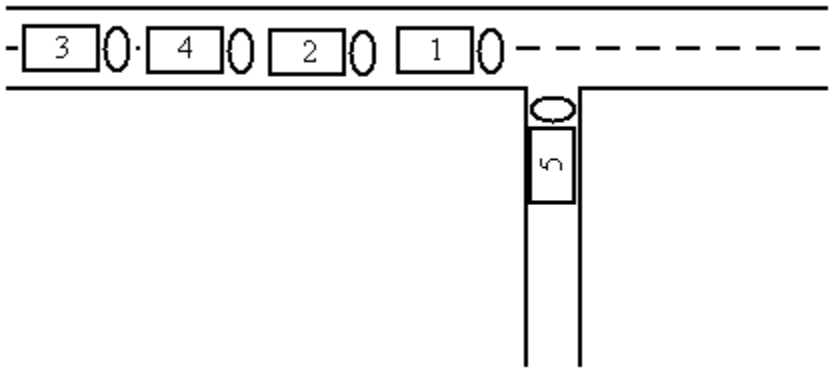
(Check the illustration for test case-1)

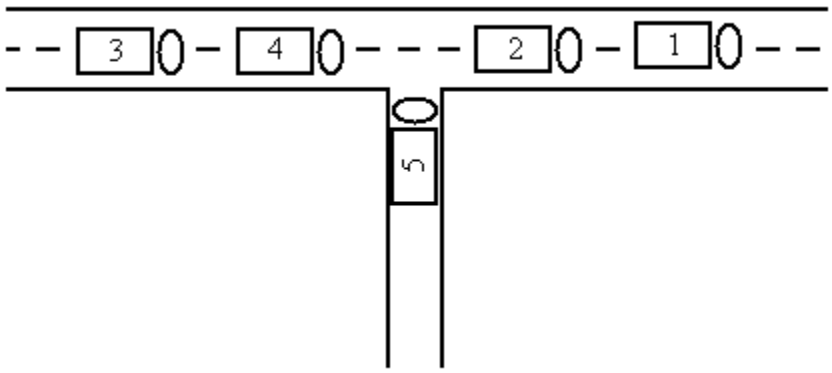
Illustration (for first test case):

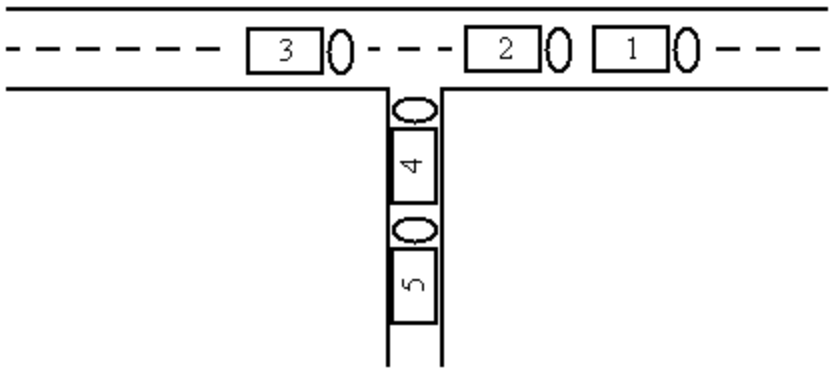
The sample input reflects the following situation:

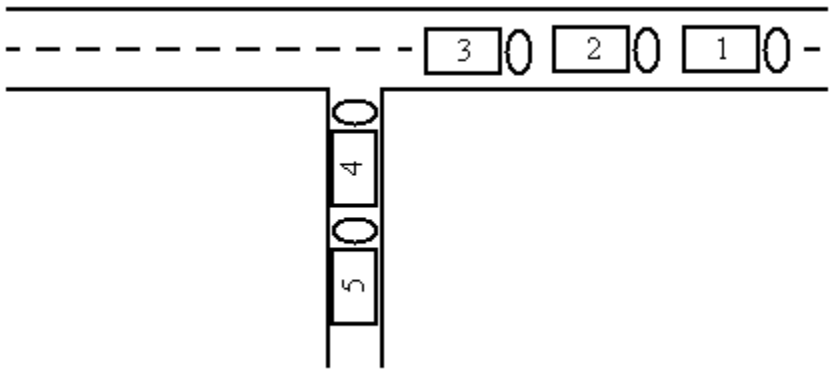


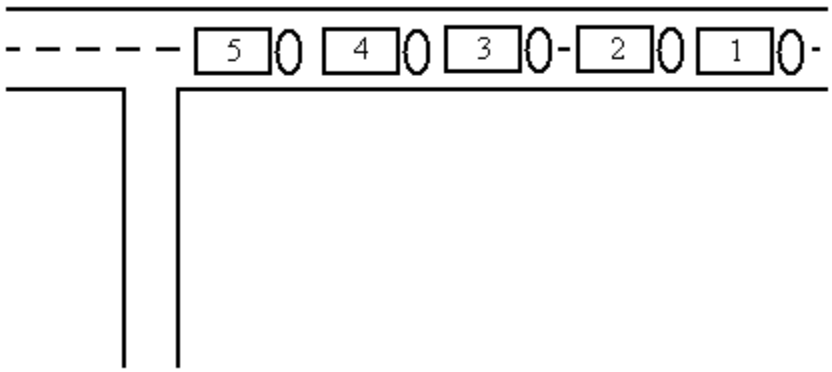
The five trucks can be reordered in the following way:






3
2
1
4
5


3
2
1
4
5


5
4
3
2
1